

Automation Template Selenium Java Cucumber

Manual de uso

Template reutilizable de automatización Web UI

v1.1.0 - Stable / Validated / Published

Hito 3 - Completed / Validated

Publicación v1.1.0: 1 de octubre de 2026

v1.2.0 - Hito 4 en desarrollo

Bloques 1 y 2 - Completed / Validated

Sin fecha de publicación de v1.2.0

Java 21 (predeterminado) | Java 17 (compatible)

Selenium 4.48.0

Cucumber 7.34.7

JUnit 5.13.4

Gradle 8.14.5

Índice

1. Inicio rápido para primera vez
2. Arquitectura
3. Stack, requisitos y VS Code
- 3.1 Configurar Java 17 paso a paso
4. Configuración y comandos
5. Primera automatización
6. Reutilización
7. Contrato CI/CD, buenas prácticas y soporte
8. CI/CD con GitHub Actions
9. Cómo ejecutar y revisar el CI/CD
10. Quality Gate, fallos y evidencias
11. Protección de main y validación de Hito 3
12. Logging y observabilidad básica
13. Evidencias automáticas
14. Glosario

Propósito

El manual explica desde cero el flujo Git, Pull Request y GitHub Actions. Para crear nuevas pruebas se necesitan conocimientos básicos de Java. El ejemplo incluido es neutral y debe sustituirse por la aplicación bajo prueba.

Información del documento

Campo	Valor
Autor / Creador del template	Israel Baz
Rol	Test Automation / Prompt Engineering / AI Automation
Mantenimiento	Israel Baz

Estado e historial de release

Versión	Fecha / estado	Cambios principales
v1.2.0	En desarrollo; sin fecha	Hito 4 en desarrollo. Bloque 1: SLF4J/Logback - Completed / Validated. Bloque 2: EvidenceManager, screenshots, attachment Cucumber y logging - Completed / Validated. No publicado. Bloques posteriores pendientes.
v1.1.0	1 de octubre de 2026	Stable / Validated / Published. Hito 3 y Bloques 1-4 Completed / Validated. CI y Quality Gate validados.
v1.0.2	25 de septiembre de 2026	Documentation-only Hotfix, Stable / Validated / Published.
v1.0.1	25 de septiembre de 2026	Hardening + CI/CD Readiness. Estado Stable / Validated / Published.
v1.0.0	17 de septiembre de 2026	First Stable Release con Java 21 predeterminado, ejemplo y manual PDF.

1. Inicio rápido para primera vez

Siga estos pasos sin modificar ningún archivo. Si un comando termina con BUILD SUCCESSFUL, se ejecutó correctamente. Si termina con BUILD FAILED, conserve el mensaje y consulte Troubleshooting antes de cambiar código.

Paso 1 - Abra PowerShell y navegue a la raíz:

```
cd C:\ruta\automation-template-selenium-java-cucumber
```

Paso 2 - Ejecute la prueba de ejemplo sin ventana visible:

```
.\gradlew.bat test -Dheadless=true
```

Paso 3 - Abra el reporte de ejecución:

Abra build/reports/cucumber/cucumber.html. El escenario abre la fixture HTML local src/test/resources/fixtures/example_page.html y valida el encabezado Example Domain. La fixture está versionada y no depende del contenido de un sitio externo; el escenario se reproduce en CI.

Paso 4 - Ejecute con navegador visible:

Ejecute .\gradlew.bat clean test sin el parámetro headless.

Cómo comprobar VS Code

Debe ver src, gradle, build.gradle y settings.gradle. Abra example_domain.feature: debe aparecer coloreado como Gherkin. Abra ExampleDomainPage.java: Java no debe mostrar imports en rojo. Si VS Code solicita un JDK, seleccione Java 21 y espere que termine "Importing Gradle project".

Lista antes de pedir ayuda

Estoy en la raíz del proyecto; java -version muestra Java 21; el navegador está instalado; la fixture local existe; la Feature termina en .feature y está bajo src/test/resources/features; cada frase Gherkin coincide con su Step; no guardé credenciales ni tokens en Git.

2. Arquitectura

El template estandariza la automatización Web UI: configuración externa, navegador, ejecución BDD, Page Object Model, evidencia y reportes. Es adecuado para proyectos que operan en Chrome o Edge. La configuración tiene prioridad JVM (-D), variable de entorno y archivo config.properties.

Arquitectura general

Capa	Responsabilidad
Feature	Describe comportamiento en Gherkin.
Step Definitions	Traduce intención a llamadas del framework.
Page Objects	Encapsula locators, esperas e interacciones.
WebDriver	Controla Chrome o Edge.
Hooks / reports	Gestiona ciclo de vida, screenshots y resultados.

Flujo de ejecución

gradlew test -> Gradle compila -> JUnit Platform descubre el Runner -> Cucumber carga Features -> Hook Before crea WebDriver -> Steps llaman Pages -> Hook After captura evidencia y cierra -> Reporte HTML.

Regla de diseño: Steps no contienen selectores; Pages no contienen aserciones de escenario; Hooks no incluyen lógica de negocio.

3. Stack, requisitos y VS Code

Tecnología	Versión	Uso
Java	21 (pred.) / 17	Lenguaje y toolchain.
Gradle Wrapper	8.14.5	Build y dependencias reproducibles.
Selenium Java	4.48.0	WebDriver.
Cucumber Java/Engine	7.34.7	BDD y Gherkin.
JUnit BOM	5.13.4	JUnit Platform y Suite.

Prerrequisitos

Instale JDK 21 (recomendado) o JDK 17 (compatible), y Chrome o Edge. Use siempre el Gradle Wrapper incluido; en Windows, `.gradlew.bat`. No requiere Gradle global. Valide con:

```
java -version
.\gradlew.bat --version
git --version
```

Apertura en Visual Studio Code

Abra PowerShell y ejecute:

```
cd C:\ruta\automation-template-selenium-java-cucumber
code .
```

Instale Extension Pack for Java, Gradle for Java, Cucumber (Gherkin) Full Support y GitLens. Espere que VS Code importe Gradle. Las Features están en `src/test/resources/features`; Pages en `src/main/java`; Steps, Hooks y Runner en `src/test/java`.

Selección de versión de Java

Java 21 es el predeterminado de la configuración actual. Java 17 es la única compatibilidad alternativa mediante la propiedad Gradle `javaVersion`. Java 18, 19, 20, 22 y cualquier otro valor distinto de 17 o 21 se rechazan explícitamente.

```
.\gradlew.bat clean test
.\gradlew.bat clean test -PjavaVersion=17
.\gradlew.bat clean test -PjavaVersion=21
.\gradlew.bat clean test -PjavaVersion=18 # rechazado
```

Gradle debe ejecutarse con JDK 17 o superior y la toolchain seleccionada debe estar instalada o disponible para Gradle.

Sección 3.1 — Configurar Java 17 paso a paso

Use Java 17 solo cuando lo requiera su proyecto. Java 21 continúa siendo el predeterminado de la configuración actual.

1. Instale un JDK 17 aprobado por su equipo. En estos ejemplos, la carpeta se representa como C:\ruta\jdk-17.

2. Abra PowerShell en la raíz del proyecto y configure la sesión actual. Esto no modifica permanentemente Windows:

```
$env:JAVA_HOME = 'C:\ruta\jdk-17'
$env:Path = "$env:JAVA_HOME\bin;$env:Path"
java -version
```

3. Confirme que `java -version` indica Java 17. En VS Code, use Ctrl+Shift+P, ejecute Java: Configure Java Runtime, seleccione JDK 17 y espere la importación Gradle.

4. Ejecute las pruebas:

```
.\gradlew.bat clean test -PjavaVersion=17
```

5. Confirme BUILD SUCCESSFUL y abra `build/reports/cucumber/cucumber.html`. Para volver a Java 21, abra una consola nueva o cambie `JAVA_HOME` a C:\ruta\jdk-21 y ejecute `-PjavaVersion=21`.

No guarde rutas de JDK ni `JAVA_HOME` dentro del repositorio. El workflow incluido usa Java 21. Si adopta Java 17 en otro proyecto, adapte y valide su CI antes de usarlo.

4. Configuración y comandos

config.properties contiene los valores por defecto. Los valores pueden reemplazarse con -D o con variables BASE_URL, BROWSER, HEADLESS, TIMEOUT_SECONDS y SCREENSHOT_ON_FAILURE. javaVersion usa -P, no -D.

```
browser=CHROME
headless=false
baseUrl=https://example.com/
timeoutSeconds=15
screenshotOnFailure=true
```

Objetivo	Comando
Limpiar	.\gradlew.bat clean
Limpiar y ejecutar	.\gradlew.bat clean test
Headless	.\gradlew.bat clean test -Dheadless=true
Chrome	.\gradlew.bat clean test -Dbrowser=CHROME
Edge	.\gradlew.bat clean test -Dbrowser=EDGE
URL	.\gradlew.bat clean test -DbaseUrl=https://su-aplicacion
Tags	.\gradlew.bat clean test -Dcucumber.filter.tags=@example
Tags headless	.\gradlew.bat clean test -Dheadless=true -Dcucumber.filter.tags=@example
Alias Cucumber	.\gradlew.bat cucumber

El smoke test incluido usa la fixture HTML local versionada y no depende de un sitio externo. baseUrl sigue disponible para los Page Objects de aplicaciones reales, pero no modifica este escenario de ejemplo. Los reports se generan en build/reports/cucumber/cucumber.html, build/reports/tests/test y build/test-results/test; el log se guarda en build/logs/automation.log y las capturas de fallos en build/evidence/screenshots sólo si falla un escenario, screenshotOnFailure está activo y WebDriver permite capturar.

5. Crear la primera automatización

1. Cree `src/test/resources/features/login.feature`:

```
# language: en
Feature: Login
  Scenario: Successful login
    Given the user opens the login page
    When the user signs in with "username" and "password"
    Then the dashboard is displayed
```

2. Cree `LoginPage.java` en `pages`. Mantenga locators privados y métodos de intención, por ejemplo `open()`, `login()` e `isDashboardVisible()`.

3. Cree `LoginSteps.java` en `steps`. Obtenga el driver mediante `DriverManager`, delegue a `LoginPage` y use aserciones `JUnit`.

4. Configure la URL con `-DbaseUrl=...` y ejecute `.\gradlew.bat test -Dheadless=true`.

5. Revise el reporte HTML. Use esperas explícitas `WebDriverWait`; nunca `Thread.sleep`.

6. Reutilización paso a paso

1. Copie o clone el template; conserve la base original sin cambios.

2. Actualice `rootProject.name` en `settings.gradle` y `group` en `build.gradle`.

3. Configure `baseUrl` con `-DbaseUrl=https://su-aplicacion`, sin secretos en Git.

4. Abra la URL manualmente y ejecute una prueba smoke en headless.

5. Cuando el smoke funcione, sustituya el ejemplo por sus Features, Pages y Steps.

6. Añada tags como `@smoke` y `@regression` para seleccionar subconjuntos.

7. Inicialice Git, revise `.gitignore`, cree una rama y abra Pull Request.

8. Revise el workflow incluido en `.github/workflows/ci.yml`: ya ejecuta pruebas headless y publica las evidencias disponibles.

Criterio de salida: el equipo puede configurar URL, ejecutar una prueba y consultar el reporte sin editar componentes compartidos.

7. Contrato CI/CD, buenas prácticas y soporte

Workflow de GitHub Actions

El workflow `.github/workflows/ci.yml` ejecuta `./gradlew clean test -Dheadless=true` con Java 21 de Temurin y el Gradle Wrapper en pull requests y pushes hacia main. En Windows, use `./gradlew.bat clean test -Dheadless=true` para la validación local. Exit code 0 es éxito; cualquier otro código debe fallar el job. El contrato permite BROWSER, BASE_URL, HEADLESS y filtros `cucumber.filter.tags`; recolecte `build/reports/cucumber/cucumber.html`, `build/reports/tests/test`, `build/test-results/test`, `build/logs/automation.log` y, sólo ante fallo, `build/evidence/screenshots`.

Flujo: checkout, Java 21 Temurin, Gradle Setup/Cache, Gradle Wrapper, pruebas headless, reportes y test-evidence. `gradle/actions/setup-gradle@v6` usa caché básica para reutilizar dependencias e información de Gradle; puede reducir trabajo repetitivo en ejecuciones posteriores. Si no hay caché (cache miss), Gradle descarga lo necesario y las pruebas continúan. El Wrapper sigue siendo el mecanismo oficial. La caché no cambia Selenium, Cucumber, Page Object Model, features, steps ni el smoke test local.

El workflow usa ubuntu-latest y permisos mínimos de lectura. Intenta publicar las rutas anteriores como test-evidence durante 14 días incluso si Gradle falla. Una ruta vacía, como screenshots en una ejecución sin fallos, se ignora. Descargue el artifact en Artifacts dentro del resumen de GitHub Actions. La evidencia no cambia el estado PASS/FAIL del job. Docker, Grid y secretos de CI/CD quedan fuera de este bloque.

Troubleshooting de Gradle

`./gradlew.bat --stop` detiene Gradle Daemons ante problemas transitorios. Para diagnóstico de dependencias use `./gradlew.bat clean test --offline` o `./gradlew.bat clean test -PjavaVersion=21 --offline`; sólo usa caché y puede fallar si faltan dependencias. Si aparece `PKIX path building failed` o `unable to find valid certification path`, revise certificado Java, proxy, inspección SSL, red y daemon; ejecute `java -version`, `./gradlew.bat --version`, `./gradlew.bat --stop` y `./gradlew.bat clean test`. No deshabilite SSL ni ignore certificados.

Regeneración del manual

`scripts/create_manual.py` requiere Python 3 y reportlab. Ejecute `python scripts/create_manual.py` desde la raíz. Genera `docs/Manual_Template_Automatizacion_Selenium_Java_Cucumber.pdf` y conserva una copia externa. `work/` es temporal, está ignorada por Git, no es parte del framework y puede eliminarse.

Buenas prácticas

Un Page Object por pantalla/componente; nombres descriptivos; configuración externa; locators estables; datos aislados; screenshots y logs como evidencia; commits pequeños; ramas, Pull Requests y revisión de código; actualización deliberada de dependencias.

Troubleshooting

Síntoma	Solución inicial
Java no reconocido	Instale/configure JDK 21 o 17 y valide <code>java -version</code> .
Wrapper no ejecuta	Ejecute desde la raíz y preserve <code>gradle/wrapper</code> .
Browser no inicia	Actualice Chrome/Edge y revise Selenium Manager.
Feature/Step no encontrado	Valide ruta <code>.feature</code> , tags, texto y paquete glue.
CI falla y local no	Compare variables, navegador, URL y artefactos.

8. CI/CD con GitHub Actions

CI (integración continua) valida automáticamente cambios antes de integrarlos. Detecta temprano errores de compilación y pruebas y ofrece al equipo un resultado compartido. CD significa entrega continua (preparar una versión para publicar) o despliegue continuo (publicarla automáticamente). Este proyecto utiliza principalmente CI; no despliega aplicaciones.

Qué ejecuta GitHub Actions

GitHub Actions lee `.github/workflows/ci.yml` cuando se abre o actualiza un Pull Request hacia main y cuando hay un push a main. Un Pull Request o PR es una propuesta de unir una rama de trabajo con otra. El workflow obtiene el código (checkout), prepara Java 21 Temurin y caché Gradle en un runner Linux ubuntu-latest, y ejecuta el Gradle Wrapper con Selenium y Cucumber en modo headless, sin ventana visible.

Flujo para principiantes

Developer -> feature branch -> commit -> push -> Pull Request hacia main -> GitHub Actions -> quality-gate -> Gradle -> Selenium + Cucumber -> PASS/FAIL -> evidencias -> revisión -> merge a main.

El workflow se ejecuta automáticamente. No hay botón de ejecución manual configurado. El merge debe esperar la revisión y un check exitoso; el ruleset activo de main exige un Pull Request, la rama actualizada y quality-gate aprobado; un fallo bloquea el merge.

9. Cómo ejecutar y revisar el CI/CD en GitHub Actions

Necesita Git, acceso al repositorio en GitHub y permiso para enviar ramas. Ejecute desde la raíz del proyecto.

1. Cree una rama separada de main: `git switch -c feature/mi-cambio`. Si ya existe: `git switch feature/mi-cambio`. Main es la línea compartida; una feature branch aísla su trabajo.
2. Modifique el proyecto, por ejemplo un Page Object y sus pruebas.
3. Revise `git status` y `git diff`. Compruebe que no haya credenciales ni archivos generados.
4. Ejecute las pruebas locales:

```
Windows PowerShell: .\gradlew.bat clean test "-Dheadless=true"
Linux/macOS: ./gradlew clean test -Dheadless=true
```

Espere BUILD SUCCESSFUL. Si ve BUILD FAILED, corrija el error.

Continuación: Pull Request y revisión de resultados

5. Guarde el cambio en Git: `git add ruta/del/archivo` y `git commit -m "Describe mi cambio"`. Un commit es una instantánea identificable.
6. Envíe la rama: `git push -u origin feature/mi-cambio` la primera vez; luego `git push`. Push copia los commits a GitHub.
7. En el repositorio de GitHub abra Pull requests -> New pull request. Seleccione main como base (destino) y feature/mi-cambio como compare (origen). Revise el contenido, agregue título y descripción y pulse Create pull request. No haga merge todavía.
8. GitHub detecta el PR y ejecuta `.github/workflows/ci.yml` automáticamente. Cada nuevo commit seguido de push a la misma rama actualiza el PR y inicia otra ejecución.

Desde el Pull Request

Abra Repositorio -> Pull requests -> su PR -> Checks; según la interfaz, los checks también aparecen en Conversation. Busque quality-gate y pulse Details. Allí verá checkout, Java 21, caché Gradle, pruebas, carga de evidencias y resultado final. Expanda un step para ver sus logs.

Desde Actions

Abra Repositorio -> Actions -> workflow CI -> ejecución del PR. Identifique rama, PR, commit, fecha, estado y duración. Abra la ejecución y el job quality-gate para inspeccionar cada step.

Estado visible	Qué significa
Success (verde)	Las verificaciones terminaron correctamente.
Failure (rojo)	Una verificación falló.
In progress (amarillo/progreso)	La ejecución todavía no termina.
Cancelled	La ejecución se canceló.
Skipped	Un step no se ejecutó por una condición.

Lea el texto del estado además del color: así puede interpretar el resultado aun sin distinguir colores.

10. ¿Qué es un Quality Gate?

Es una puerta de control: las verificaciones automáticas deben terminar bien antes de considerar listo un cambio. El job/check estable se llama quality-gate. Gradle devuelve exit code 0 cuando compilación y pruebas pasan; un código distinto de cero indica fallo. GitHub muestra PASS/Success para continuar a revisión o FAIL/Failure para corregir. La carga posterior de artifacts no cambia FAIL a PASS.

¿Qué hacer si el Quality Gate falla?

1. Abra el PR -> Checks -> quality-gate -> Details.
2. Localice el step Failure, expándalo y lea el error y el resumen final.
3. Descargue test-evidence si existe; revise reportes, XML, logs y screenshots. Si Gradle terminó antes de generar archivos, algunos faltarán.
4. Corrija el problema en su rama y repita la prueba local:

```
.\gradlew.bat clean test "-Dheadless=true"
```

En Linux/macOS use `./gradlew clean test -Dheadless=true`.

5. Revise git diff, haga otro git add y git commit, y ejecute git push. El PR se actualiza y Actions ejecuta de nuevo quality-gate. Espere el nuevo resultado.

Cómo descargar las evidencias de GitHub Actions

Un artifact es un archivo descargable creado por una ejecución y separado del código. Abra Repository -> Actions -> CI -> ejecución del PR -> Artifacts -> test-evidence. Descargue y extraiga el ZIP. Se conserva 14 días. Se intenta publicar tanto en PASS como en FAIL, pero sólo aparece cuando existen archivos. Si falta, revise el step Upload test evidence y si Gradle creó resultados. Las capturas suelen faltar cuando no fallan escenarios.

Ruta local / artifact	Qué contiene
build/reports/cucumber/cucumber.html	Escenarios y pasos Cucumber.
build/reports/tests/test/	Reporte HTML Gradle/JUnit.
build/test-results/test/	Resultados XML estructurados.
build/logs/automation.log	Log cronológico.
build/evidence/screenshots/	Capturas de fallos cuando correspondan.

11. Protección de la rama main

Branch Protection es una regla de GitHub que limita cómo se integra código en main, la línea compartida. El flujo recomendado es feature branch -> Pull Request -> quality-gate -> code review -> merge. Un status check es el resultado visible del job; si es Required Status Check, un fallo o estado pendiente bloquea el merge.

El ruleset activo de main requiere Pull Request, quality-gate aprobado y rama actualizada antes del merge. Bloquea force push y restringe la eliminación de main. Se consulta en Repository -> Settings -> Rules -> Rulesets. El workflow ejecuta el check; el ruleset aplica la protección.

Ejemplo completo

Ana crea feature/ajuste-page, modifica un Page Object, ejecuta pruebas locales, hace commit y push y abre un PR hacia main. Actions inicia quality-gate. Si aparece Success, otra persona revisa el código y se podrá hacer merge cuando se cumplan las reglas. Si falla una prueba, Ana abre Details, descarga test-evidence, corrige, prueba localmente, hace un nuevo commit y push. El PR y el check se actualizan automáticamente.

Validación real del Quality Gate

En GitHub se comprobó la secuencia PASS inicial -> FAIL controlado -> bloqueo del merge por quality-gate requerido -> test-evidence disponible -> restauración de la aserción -> PASS final -> merge -> PASS post-merge en main. Los artifacts se publicaron en ejecuciones PASS y FAIL.

Estado del Hito 3 / v1.1.0

Bloque 1: CI base - Completed / Validated. Bloque 2: evidencias - Completed / Validated. Bloque 3: caché Gradle - Completed / Validated. Bloque 4: Quality Gate, protección de main y documentación CI/CD - Completed / Validated. Hito 3: Completed / Validated. v1.1.0: Stable / Validated / Published. Fecha de publicación: 1 de octubre de 2026.

Auditoría de versionado y release

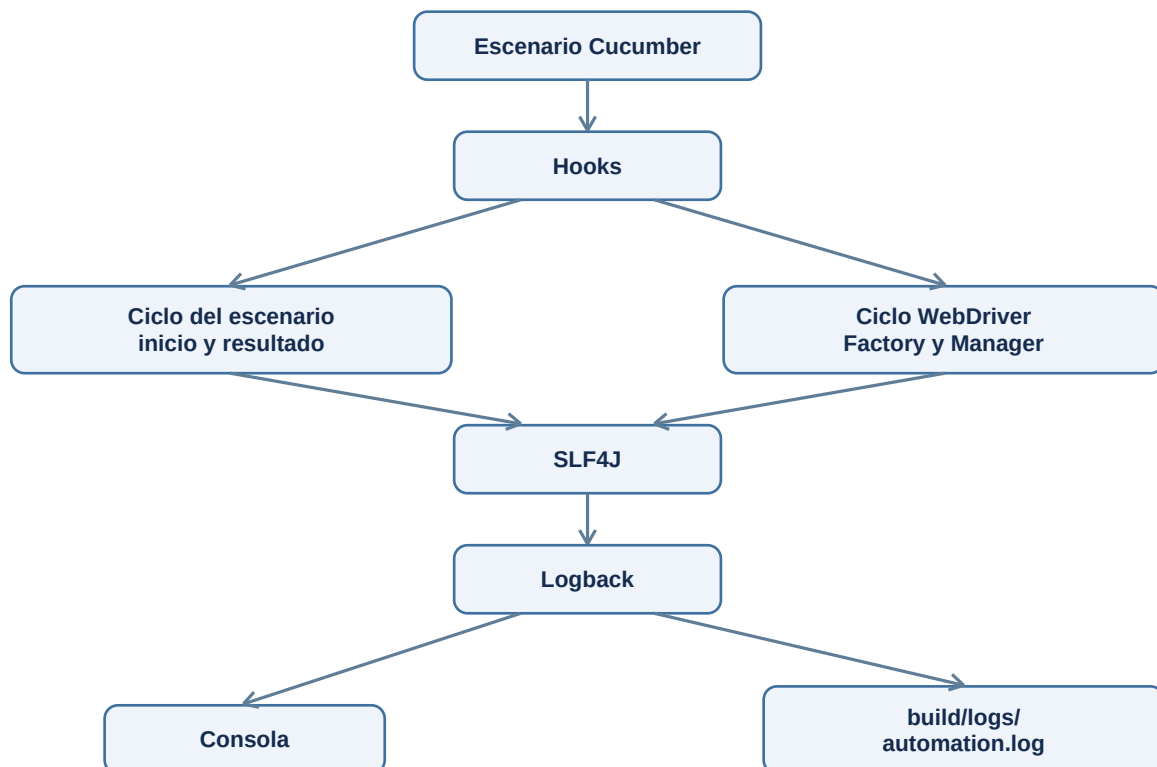
Antes del cierre formal de cada Hito, defina la fecha oficial de publicación y compruebe que coincida en portada, historial, README bilingües, guía, changelog, versioning, release process y PDF. La portada, el historial y los resúmenes finales muestran versión, Stable / Validated, Hito Completed / Validated y fecha oficial; no deje la fecha vacía, TBD o Pending. La publicación es un evento posterior: tras el tag y GitHub Release, el historial podrá incorporar Published.

12. Logging y observabilidad básica

Hito 4 - Reporting + Logging / Observability, en desarrollo. v1.2.0 sigue en desarrollo y no tiene fecha de publicación. Bloque 1 logging y Bloque 2 evidencias automáticas: Completed / Validated. Bloques posteriores pendientes.

El logging ayuda a reconstruir el inicio, la configuración efectiva, el resultado y el cierre de un escenario cuando una prueba falla o se ejecuta en CI/CD. El código usa SLF4J 2.0.20 como API común para Hooks y driver, evitando configurar salidas en cada clase. Logback 1.6.5 es el proveedor de pruebas que centraliza niveles, formato y destinos en `src/test/resources/logback-test.xml`.

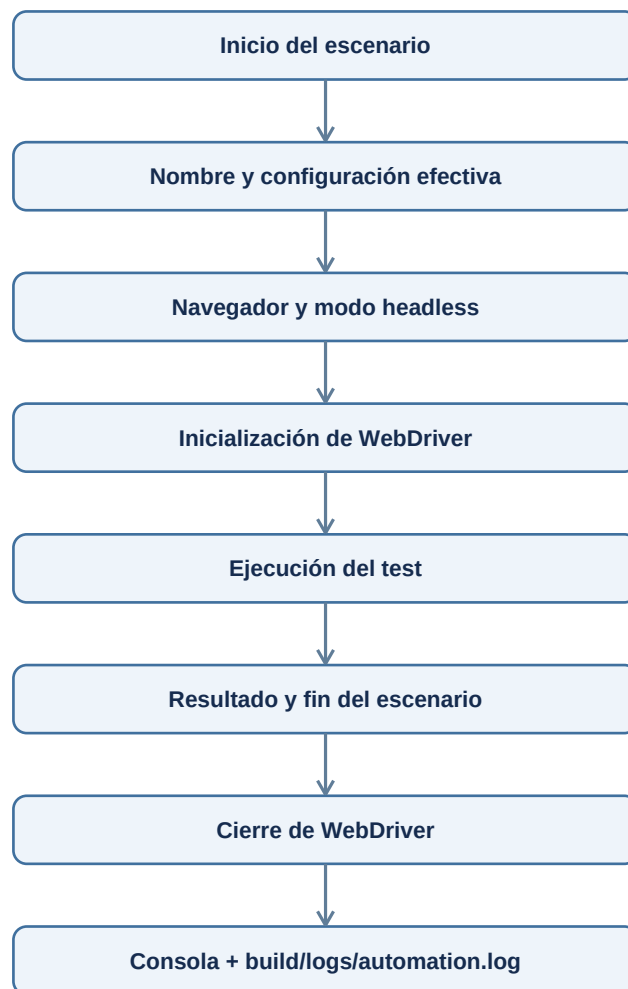
Diagrama A - Arquitectura de logging



Los eventos del framework pasan por SLF4J y Logback. Cada evento se emite a consola y a `build/logs/automation.log`; no se duplican llamadas de logging en Steps o Page Objects.

12. Flujo de observabilidad

Diagrama B - Eventos en orden de ejecución



Los Hooks registran inicio, nombre, navegador y headless efectivos antes de crear el driver. DriverFactory registra la inicialización; al finalizar, el Hook registra el estado Cucumber y DriverManager cierra el WebDriver. La consola y el archivo reciben los eventos a medida que ocurren.

Niveles de logging

Nivel	Uso
DEBUG	Detalle técnico de creación del driver; oculto por defecto.
INFO	Eventos normales: escenario, configuración segura y WebDriver.
WARN	Condición recuperable, como ausencia de soporte para capturas.
ERROR	Fallo al cerrar WebDriver o manejar evidencia existente.

12. Ejecución, seguridad y alcance

Ejecute `.\gradlew.bat clean test -Dheadless=true` en Windows, o `./gradlew clean test -Dheadless=true` en GitHub Actions. Gradle muestra los logs de test en consola. Logback guarda el archivo bajo `build/`, ignorado por Git. `gradlew clean` elimina el log anterior y la siguiente ejecución crea uno nuevo. El workflow existente ya incluye `build/logs/` en el artifact `test-evidence`; no se modificó el workflow.

Extracto abreviado de una ejecución headless real

```
2026-10-01 17:36:35.726 INFO [Test worker] Hooks - Starting scenario: Validar el encabezado de la
página de ejemplo
2026-10-01 17:36:35.734 INFO [Test worker] Hooks - Scenario configuration: browser=CHROME,
headless=true
2026-10-01 17:36:37.110 INFO [Test worker] Hooks - Finished scenario: Validar el encabezado de la
página de ejemplo, status=PASSED
2026-10-01 17:36:37.208 INFO [Test worker] DriverManager - WebDriver closed.
```

No registre secretos, tokens, contraseñas, cookies, headers confidenciales ni propiedades indiscriminadamente. `baseUrl` no se registra porque una URL arbitraria puede incluir datos sensibles. Las rutas de error existentes registran excepciones completas; las de herramientas externas podrían contener datos de la aplicación. Revise las trazas antes de compartir logs o artifacts.

Si falta `automation.log`, confirme que test inició el proceso de pruebas y consulte `build/logs/`. Si falta `DEBUG`, revise el nivel `INFO` predeterminado en `logback-test.xml`. Si hay avisos de proveedores `SLF4J`, inspeccione el classpath con `.\gradlew.bat dependencies --configuration testRuntimeClasspath`. Si falla la descarga por `PKIX`, revise certificados y proxy del JDK sin desactivar TLS.

Alcance del Hito 4

El Bloque 1 (Completed / Validated) estableció `SLF4J` y `Logback`. El Bloque 2 (Completed / Validated) incorporó `EvidenceManager`, `screenshots`, `attachment Cucumber` y `logging`. Los bloques posteriores siguen pendientes. `v1.2.0` continúa en desarrollo, sin fecha y no publicada.

13. Evidencias automáticas ante fallos

Hito 4, Bloque 2 - Completed / Validated. El Hook @After detecta el fallo Cucumber y, si screenshotOnFailure=true, pasa WebDriver a EvidenceManager antes del cierre. El gestor comprueba la sesión y TakesScreenshot. Si falta el driver o falla la captura, registra un warning sin sustituir el error original.

Flujo de evidencia

Paso	Resultado
1. Escenario y Hook @After	Detecta si el escenario falló y la captura está activada.
2. WebDriver disponible	Comprueba sesión activa y TakesScreenshot.
3. Screenshot	Obtiene los bytes PNG.
4. Archivo PNG	Guarda en build/evidence/screenshots/.
5. Attachment	Adjunta image/png al escenario Cucumber.
6. Log	Registra resultado mediante SLF4J y Logback.
7. Driver quit	Cierra y limpia el WebDriver al final.

El nombre contiene hasta 80 caracteres seguros del escenario, timestamp y UUID. build/ está ignorado por Git. El log registra intento, ruta y attachment sin bytes ni Base64. Un escenario exitoso no genera captura de fallo.

Para validar: ejecute el smoke test y compruebe ausencia de screenshots; cambie temporalmente la aserción local, repita y revise PNG, reporte Cucumber y automation.log. Restaure la aserción inmediatamente y ejecute la suite hasta BUILD SUCCESSFUL. Repita con headless=false. Revise datos sensibles antes de compartir capturas.

14. Glosario

Términos clave del template y de la automatización Web UI.

Término	Definición
Automation Testing	Ejecución de pruebas mediante software para validar una aplicación.
Selenium	Biblioteca que automatiza navegadores web.
WebDriver	Interfaz que controla Chrome o Edge desde el código.
Cucumber	Herramienta BDD que ejecuta escenarios escritos en Gherkin.
BDD	Enfoque que describe el comportamiento esperado antes de implementarlo.
Gherkin	Lenguaje legible para escribir Features, Scenarios y Steps.
Feature	Archivo o capacidad de negocio descrita en Gherkin.
Scenario	Caso concreto que valida un comportamiento esperado.
Step Definition	Código Java que implementa una frase de Gherkin.
Page Object Model	Patrón que encapsula la UI y sus interacciones en clases Page.
Hooks	Código que prepara y limpia el estado antes o después de escenarios.
Runner de Cucumber	Clase que configura el descubrimiento y la ejecución de Cucumber.
Gradle	Herramienta de compilación, dependencias y ejecución de pruebas.
Gradle Wrapper	Scripts versionados que ejecutan la versión requerida de Gradle.
JUnit Platform	Plataforma que descubre y coordina la ejecución de pruebas.
Headless	Ejecución del navegador sin mostrar una ventana gráfica.
CI/CD	Integración y entrega continua mediante una pipeline automatizada.
Pipeline	Secuencia automatizada de compilación, pruebas y publicación.
Git	Sistema de control de versiones distribuido.
GitHub Enterprise	Plataforma corporativa para repositorios, revisiones y pipelines.
Locator	Regla que permite identificar un elemento en la interfaz.
XPath	Sintaxis para ubicar elementos a partir de la estructura del DOM.
CSS Selector	Sintaxis para ubicar elementos con atributos o clases CSS.
Assertion	Verificación que confirma el resultado esperado de una prueba.
Test Data	Datos controlados utilizados durante la ejecución de pruebas.
Environment Variable	Valor externo al código que configura una ejecución.
Explicit Wait	Espera controlada hasta que una condición de WebDriver se cumple.

Glosario (continuación)

Término	Definición
Screenshot	Captura visual utilizada como evidencia, especialmente ante fallos.
CI	Integración continua: valida cambios automáticamente antes de integrarlos.
CD	Entrega o despliegue continuo; aquí no hay despliegue automático.
GitHub Actions	Servicio de GitHub que ejecuta workflows.
Workflow	Archivo de instrucciones automáticas, como ci.yml.
Job	Grupo de pasos ejecutado en un runner.
Step	Instrucción individual de un job.
Runner de GitHub Actions	Equipo temporal que ejecuta un job.
ubuntu-latest	Etiqueta de un runner Linux Ubuntu en GitHub Actions.
Pull Request	Propuesta de integrar una rama después de revisión.
Branch	Línea de trabajo separada dentro de Git.
main	Rama principal compartida del repositorio.
Feature Branch	Rama dedicada a un cambio concreto.
Quality Gate	Verificación que debe pasar antes de considerar listo un cambio.
Status Check	Resultado visible de una verificación de GitHub.
Required Status Check	Check que debe pasar para permitir el merge.
Branch Protection	Reglas que protegen una rama ante integraciones indebidas.
Artifact	Archivo descargable producido por una ejecución.
Log	Registro cronológico de mensajes de ejecución.
Report	Resumen legible de resultados de pruebas.
Exit Code	Número de salida: 0 éxito; otro valor, fallo.
PASS	La verificación terminó correctamente.
FAIL	La verificación terminó con un error.
Commit	Instantánea identificable de cambios en Git.
Push	Envío de commits locales al repositorio remoto.
Merge	Unión de cambios de una rama a otra.
Checkout	Obtención del código de una revisión para ejecutarlo.